```
const modelWithParser = model.withStructuredOutput(format, {
  method: 'jsonMode',
});
const result = await modelWithParser.invoke('Tell me a poem');
Error: ChatGoogleGenerativeAI only supports "functionCalling" as a method.
```

This error indicates that the `ChatGoogleGenerativeAI` model does not support JSON mode for structured outputs and can only operate with the function calling method. The fix here is to just remove the config option for `method: 'jsonMode'`. To arrive at that conclusion, we had to review the implementation of the `ChatGoogleGenerativeAI` class.

When faced with such errors, it can be challenging to determine the exact capabilities of each provider. Therefore, a practical approach is to verify the implementation details of the provider by examining the actual API calls made within the code. This allows you to understand what features are supported and how to work around limitations effectively.

For further verification, you may need to explore the examples provided in the LangChain.js repository. The examples folder contains dozens of working implementations that demonstrate correct usage of each provider and reference function call. This is all expected, since there are many ways of utilizing LangChain.js, and not all of them are properly documented on the official documentation site, so experimentation is necessary.

Of course, sometimes certain features are not entirely supported for a specific model, whether it is intentional or not, and you will have to provide alternatives.

8.4 Testing strategies for AI applications

There are four areas to consider when testing AI-powered web apps: unit and integration testing, mocking LLM responses, and testing Vercel AI SDK and LangChain.js responses, shown in figure 8.8.

8.4.1 Unit and integration testing in React and Next.js

In web applications built with React and Next.js that use generative AI, we write tests to ensure both the individual components and the overall system function correctly. There are two main testing methods: unit testing and integration testing.

> **Project code: chat-testing**
>
> The code for this example project is in the ch08/chat-testing folder. You can run the test cases of this example by running this command in the root folder of the project repository:
>
> ```
> $ npm run test -w ch08/chat-testing
> ```
>
> You won't need either the OpenAI API or the Google API keys since the code listed is for testing purposes.